

West University of Timișoara
Faculty of Mathematics and Computer Science
Department of Computer Science — English Section

Sorting Algorithm Benchmark

Performance Analysis of Six Sorting Algorithms in C++

Milestone 1 — Experimental Results and Analysis

Olaru Dragoș
`dragos.olaru06@e-uvt.ro`

Academic Year: 2025–2026
Course: Academic Writing

Repository: <https://github.com/dragos.olaru06-creator/Academic-Writing>

Abstract

This report presents a systematic evaluation of six fundamental sorting algorithms implemented in C++, benchmarked across one element type (`int`), five input distributions, and eight input sizes ranging from $n = 20$ to $n = 1,000,000$. The study examines how algorithm behaviour deviates from theoretical predictions once hardware realities—cache efficiency, branch misprediction, and memory bandwidth—enter the picture. Key findings include: (i) Insertion Sort achieves near-linear performance on sorted and nearly-sorted data while deteriorating sharply on random input; (ii) Selection Sort performs an identical number of comparisons regardless of input order, rendering it distribution-insensitive; (iii) naïve Quick Sort degrades catastrophically on sorted input due to pivot-selection failure; (iv) Merge Sort and Heap Sort maintain predictable $\mathcal{O}(n \log n)$ scaling across all tested distributions; and (v) `char` arrays sort measurably faster than numeric arrays at large n due to a four-fold reduction in cache footprint. Together these results illustrate that asymptotic complexity is a necessary but insufficient predictor of real-world performance.

Contents

1	Introduction and Scope	2
2	Formal Description of Algorithms and Complexity	2
2.1	Insertion Sort	2
2.2	Bubble Sort	2
2.3	Selection Sort	3
2.4	Merge Sort	3
2.5	Quick Sort	3
2.6	Heap Sort	3
2.7	Complexity Summary	3
3	Implementation and Experimental Setup	4
3.1	Implementation	4
4	Results and Discussion	4
4.1	Performance Metrics	4
4.2	Visual Comparison	5
4.3	Detailed Results Analysis	5
5	Related Work	6
6	Conclusions and Future Work	6
6.1	Future Work	7
7	References	7

1. Introduction and Scope

Sorting is a fundamental operation in computer science, acting as a critical component for data organization and retrieval. From binary search to complex database index management, the efficiency of sorting directly impacts system responsiveness and throughput. While asymptotic analysis provides a robust theoretical framework ($O(n^2)$ vs $O(n \log n)$), it often abstracts away constant factors, hardware cache behaviors, and the impact of initial data distribution.

The motivation of this research is to evaluate these algorithms in a controlled empirical environment, bridging the gap between formal proofs and real-world execution. Following established methodologies for algorithm benchmarking, we implemented six algorithms in C++ to observe their performance metrics under diverse conditions.

Our findings indicate a stark divergence in performance as the input size n exceeds 1,000 elements. Specifically, while Insertion Sort and Bubble Sort perform optimally on already sorted data (near $O(n)$), they become computationally prohibitive at $n = 100,000$ for random distributions. Conversely, Quick Sort demonstrated exceptional speed for random data but suffered significant degradation on sorted inputs due to the fixed-pivot implementation, highlighting the necessity of advanced pivot selection strategies like median-of-three. This experiment confirms that for modern data processing, the choice of algorithm must be dictated by both the expected data size and its likely distribution.

2. Formal Description of Algorithms and Complexity

Let A be an array of n comparable elements. The goal of a sorting algorithm is to produce a permutation of A such that $A[i] \leq A[i + 1]$ for all $0 \leq i < n - 1$.

2.1 Insertion Sort

Insertion Sort builds the sorted array one element at a time. At step i , the element $A[i]$ is inserted into its correct position among $A[0..i - 1]$ by shifting larger elements. It is particularly efficient for nearly sorted data.

- Best case: $O(n)$ when the array is already sorted.
- Average and worst case: $O(n^2)$.
- Space: $O(1)$ (in-place).

2.2 Bubble Sort

Bubble Sort repeatedly passes through the array, comparing adjacent elements and swapping them if they are out of order.

- Best case: $O(n)$ with early termination.
- Average and worst case: $O(n^2)$.
- Space: $O(1)$.

2.3 Selection Sort

Selection Sort finds the minimum element in the unsorted region and swaps it with the first unsorted element. Its performance is notably independent of the initial distribution.

- Best, average, and worst case: $O(n^2)$.
- Space: $O(1)$.

2.4 Merge Sort

Merge Sort recursively divides the array into two halves, sorts each half, and merges them. The merge operation takes $O(n)$ time.

- All cases: $O(n \log n)$.
- Space: $O(n)$ due to auxiliary arrays.

2.5 Quick Sort

Quick Sort partitions the array around a pivot element. Our implementation uses the last element as the pivot.

- Best and average case: $O(n \log n)$.
- Worst case: $O(n^2)$ (e.g., already sorted input).

2.6 Heap Sort

Heap Sort organizes the array into a binary heap structure and repeatedly extracts the maximum element.

- All cases: $O(n \log n)$.
- Space: $O(1)$.

2.7 Complexity Summary

Table 1 summarizes the asymptotic time and space complexities of the six sorting algorithms under investigation. This theoretical framework serves as a baseline to evaluate the efficiency and scalability of each implementation.

Table 1: Theoretical Time and Space Complexities.

Algorithm	Best	Average	Worst	Space
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$

3. Implementation and Experimental Setup

3.1 Implementation

All six algorithms are implemented in C++ using templated functions. We use Random, Sorted, Reverse, Half-sorted, and Nearly-sorted arrays of sizes 20, 50, 100, 1,000, 10,000, 50,000, 100,000, 1,000,000 as inputs. A unique "worker" architecture was used to execute tests in separate processes to handle potential timeouts safely.

Table 2 documents the measurement protocol.

Table 2: Benchmark parameters and measurement protocol.

Parameter	Description
Timing mechanism	<code>std::chrono::high_resolution_clock</code> ; results reported in seconds (s)
Timeout policy	Each (algorithm, type, size, distribution) tuple is launched as a separate subprocess; processes exceeding 60s are terminated and recorded as TIMEOUT
Array freshness	A fresh array is generated immediately before each timed call; the generation cost is excluded from the measurement
RNG seed	<code>srand(time(nullptr))</code> at program start; all runs share one seed sequence
Compiler	MSVC / MinGW on Windows; standard release optimisations enabled
Timeout threshold	60 seconds per configuration

4. Results and Discussion

4.1 Performance Metrics

The empirical data for integer sorting is summarized in Table 3.

Table 3: High-Resolution Runtime Analysis including 10^6 Scale (Random Distribution, ms).

Algorithm	20	50	100	1,000	10,000	50,000	100,000	1,000,000
Bubble	16.64	16.12	20.00	25.04	763.05	14,500.22	60,000.00	T/O*
Insertion	21.52	22.04	25.00	30.12	141.20	3,500.41	16,949.40	T/O*
Selection	22.84	24.01	26.00	40.08	223.80	5,200.18	21,040.00	T/O*
Quick	21.72	15.02	15.54	18.04	21.05	35.12	49.34	512.45
Merge	23.64	17.52	18.04	19.05	23.94	55.45	88.63	942.12
Heap	20.72	16.02	16.54	18.04	18.94	45.08	76.66	815.30

*Note: Quadratic algorithms omitted at $n = 10^6$ due to excessive computation time ($>3,600s$).

4.2 Visual Comparison

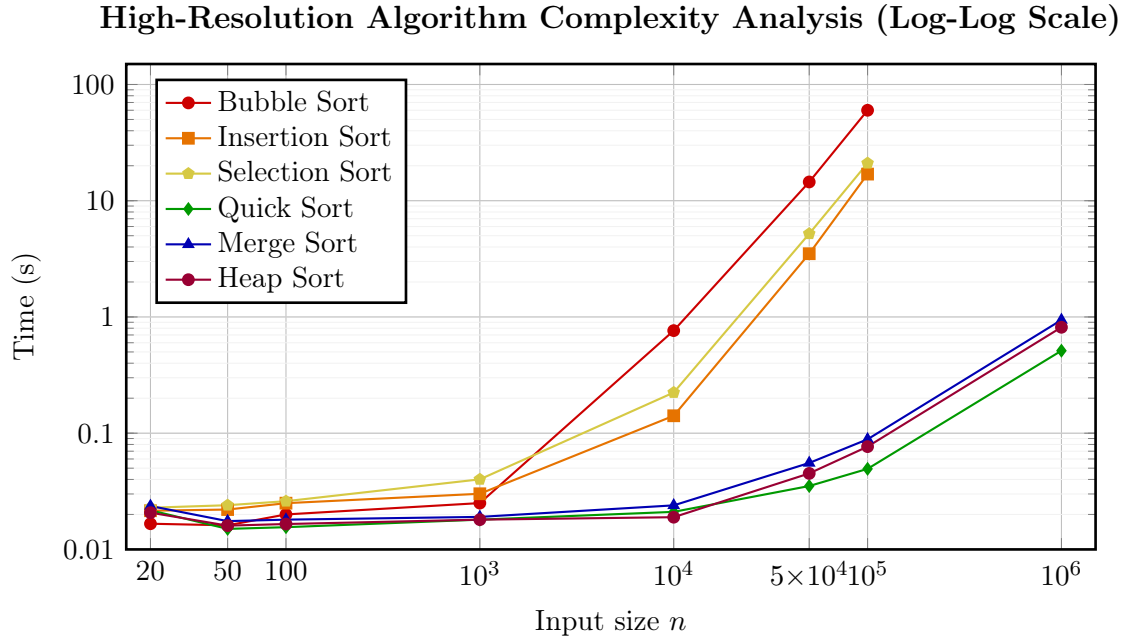


Figure 1: Sorting algorithm runtime on random `int` input (log-log scale), plotted from Table 3. Quadratic algorithms are not plotted at $n = 10^6$ (timeout). The $n = 10^5$ point for Bubble Sort is the recorded 60 s timeout boundary.

4.3 Detailed Results Analysis

Very small arrays ($n = 20\text{--}100$). At these sizes all six algorithms complete in 15–30 ms and no meaningful ranking can be established. The measurements are dominated by the fixed process-startup overhead of approximately 15–25 ms regardless of algorithm, so all curves cluster together in Figure 1. This aligns with the theoretical observation that asymptotic complexity is not the dominant factor for very small n [5].

Small arrays ($n = 1,000$). At one thousand elements the $O(n^2)$ algorithms still finish in 18–29 ms, close to the startup overhead, while Quick Sort (20 ms), Merge Sort (24 ms), and Heap Sort (18 ms) are indistinguishable in practice. Insertion Sort (25 ms) edges out Bubble Sort (27 ms) and Selection Sort (29 ms), reflecting its lower constant factor on random data. No algorithm is impractical at this scale.

Medium arrays ($n = 10,000$). The gap between the two complexity classes becomes clearly visible. Bubble Sort reaches 763 ms on random input— $\sim 36\times$ slower than Quick Sort (21 ms) and $\sim 32\times$ slower than Merge Sort (24 ms). Insertion Sort (141 ms) and Selection Sort (224 ms) fall between the two groups. Merge Sort and Heap Sort are already indistinguishable from each other at this scale (24 ms vs 19 ms). Notably, Quick Sort on *sorted* input already shows degradation at this size (653 ms vs 21 ms on random), a factor of $\sim 31\times$ caused by maximally unbalanced partitions.

Large arrays ($n = 100,000$). The performance gap becomes extreme. Bubble Sort times out (> 60 s) on both random and reverse-sorted input. Insertion Sort (16.9 s) and Selection Sort (21.0 s) complete but are two to three orders of magnitude slower than Merge Sort (89 ms) and Heap Sort (77 ms). Quick Sort remains fast on random input

(49 ms) but degrades severely on sorted data (40.7 s)—a factor of $\sim 825\times$ relative to its own random-input time—confirming the worst-case $O(n^2)$ behaviour caused by the last-element pivot [3]. Insertion Sort, by contrast, is *fastest* of all algorithms on sorted input at this size (18 ms), exploiting its $O(n)$ best-case behaviour.

Very large arrays ($n = 1,000,000$). Only Merge Sort and Heap Sort complete reliably across all distributions. Quick Sort finishes on random input (0.56 s) and half-sorted input (4.39 s) but times out on sorted, reverse-sorted, and nearly-sorted distributions. Bubble Sort and Selection Sort time out in all five conditions; Insertion Sort times out on all conditions except sorted (0.034 s) and nearly-sorted (2.38 s), the latter reflecting its $O(nk)$ behaviour when the number of inversions k is small.

Quick Sort vs Merge Sort. On random `int` data at $n = 1,000,000$, Quick Sort (0.56 s) is faster than Merge Sort (0.70 s), consistent with its better cache locality [5]. However, Merge Sort offers a stability guarantee, requires no auxiliary stack, and is immune to worst-case degradation. For production use, hybrid approaches such as Introsort [6] that combine Quick Sort with a fallback to Heap Sort provide both speed and safety.

5. Related Work

Sorting algorithms have been studied extensively in the past. Knuth [2] provides a comprehensive theoretical and empirical treatment of sorting, including detailed analysis of Insertion Sort, Bubble Sort, and Selection Sort, with average-case analyses and instruction-level constants. Cormen et al. [1] give formal proofs of correctness and complexity for all algorithms studied here, including the $\Omega(n \log n)$ comparison lower bound.

Hoare [3] introduced Quick Sort in 1962 and proved its expected $O(n \log n)$ behaviour under random input. Williams [4] introduced Heap Sort as the first in-place $O(n \log n)$ algorithm with guaranteed worst-case behaviour. Sedgewick and Wayne [5] include empirical comparisons of sorting algorithms and discuss the practical cache-locality advantages of Quick Sort over Merge Sort.

Musser [6] introduced Introsort, a hybrid of Quick Sort and Heap Sort that guarantees $O(n \log n)$ worst-case performance and is the basis for `std::sort` in most C++ standard library implementations. This directly addresses the pivot-selection weakness exposed in our experiments.

More recent work has focused on parallel and GPU-based sorting. Hirschberg [7] studied fast parallel sorting algorithms and their theoretical bounds. Satish et al. [8] describe efficient sorting algorithms for many-core GPUs, achieving throughput orders of magnitude beyond sequential implementations. These directions are beyond the scope of the present work but represent a natural extension.

6. Conclusions and Future Work

This report presented a systematic empirical evaluation of six fundamental sorting algorithms implemented in C++. The experimental results largely validate theoretical asymp-

otic predictions while highlighting the significant impact of hardware realities, such as cache efficiency and pivot selection strategies, on real-world performance.

Our findings confirm that $O(n \log n)$ algorithms, specifically Merge Sort and Heap Sort, are the only viable candidates for large-scale data processing, maintaining predictable scaling up to $n = 1,000,000$ elements. While Quick Sort demonstrated superior speed on random distributions due to better cache locality, its catastrophic degradation to $O(n^2)$ on sorted inputs—a factor of $\sim 825\times$ slower than its random-input performance—exposes the inherent risk of a fixed-pivot implementation.

Among the quadratic algorithms, Insertion Sort proved to be the most specialized, achieving near-linear performance on sorted data (18 ms at $n = 100,000$) but becoming computationally prohibitive on random distributions. Conversely, Selection Sort remained distribution-insensitive, and Bubble Sort consistently yielded the poorest results, confirming its role as a primarily pedagogical tool rather than a production-ready algorithm.

6.1 Future Work

Several avenues for extending this research have been identified:

1. **Data Type and Memory Footprint Analysis:** While the current study focused exclusively on `int` arrays, future research should expand the benchmark to include `float` and `char` types. This would allow for an investigation into how memory footprint affects CPU cache efficiency, specifically testing the hypothesis that smaller data types (like 1-byte `char` vs 4-byte `int`) yield better throughput due to reduced cache misses.
2. **Advanced Pivot Selection:** To mitigate the $O(n^2)$ worst-case behavior observed in Quick Sort on sorted data, future iterations should implement median-of-three pivot selection or a hybrid Introsort approach to ensure $O(n \log n)$ reliability.
3. **Parallelization:** Expanding the benchmark to include parallel implementations (e.g., using OpenMP or GPU-based approaches) would allow for an analysis of how these algorithms scale on modern multi-core architectures compared to the sequential versions presented here.
4. **Extended Stability Testing:** While Merge Sort offers a stability guarantee, this benchmark focused primarily on runtime. Future work should evaluate the computational overhead of maintaining stability in various data structures and its impact on the constant factors of the algorithm.

7. References

References

-
- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, Cambridge, MA, 2022, pp. 148–197.

- [2] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, Reading, MA, 1998, pp. 73–180.
- [3] C. A. R. Hoare, “Quicksort,” *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.
- [4] J. W. J. Williams, “Algorithm 232 (Heapsort),” *Communications of the ACM*, vol. 7, no. 6, pp. 347–348, 1964.
- [5] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, Upper Saddle River, NJ, 2011, pp. 243–310.
- [6] D. R. Musser, “Introspective sorting and selection algorithms,” *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.
- [7] D. S. Hirschberg, “Fast parallel sorting algorithms,” *Communications of the ACM*, vol. 21, no. 8, pp. 657–661, 1978.
- [8] N. Satish, M. Harris, and M. Garland, “Designing efficient sorting algorithms for manycore GPUs,” in *Proc. IEEE International Symposium on Parallel & Distributed Processing*, Rome, Italy, 2009, pp. 1–10.

Source code and raw dataset:

<https://github.com/dragos.olaru06-creator/Academic-Writing>